

Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет програмних систем та прикладної математики
Кафедра системного програмування і спеціалізованих комп’ютерних систем

ЛАБОРАТОРНА РОБОТА №3

з дисципліни

“Автоматизоване проєктування КС”

Тема: «Побудова моделі синхронного багатофункціонального регістру»

Виконав: Ляшенко Д.Д.
Студент групи КВ-33

Київ 2026

План виконання лабораторної роботи:

1. Описати модель синхронного багатофункціонального регістру (лічильника) використовуючи мову A-HDL.
2. Побудувати часові діаграми роботи пристрою.
3. Довести працездатність моделі на всіх наборах керуючих інструкцій.

Варіант завдання:

Варіант	Перетворення в код	Зсув ліворуч	Зсув праворуч	Арифметична операція	Логічна операція	Додаткова функція
09	Інверсний	Цик.	Лог.	+In	nand	Розворот

Розрядність регістру – 4 біти, затримка розповсюдження сигналу – 100 ps, обов'язкові операції – збереження та запис. Арифметична та логічна операції виконуються над числом, що зберігається в регістрі, і числом, що надходить на вхід DataIn.

Хід виконання лабораторної роботи:

Система команд регістру (код команди – вхід Command):

Код	Команда	Виконувана дія над private
000	Збереження	private := private (стан не змінюється)
001	Запис	private := DataIn
010	Перетворення в код (інверсний)	private := not private
011	Зсув ліворуч (циклічний)	private := private(2 downto 0) & private(3)
100	Зсув праворуч (логічний)	private := '0' & private(3 downto 1)
101	Арифметична операція (+In)	private := private + DataIn
110	Логічна операція (nand)	private := private nand DataIn
111	Додаткова функція (розворот)	private := private(0) & private(1) & private(2) & private(3)

LAB3:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity lab3 is
    generic (Delay : time := 100 ps);       -- затримка розповсюдження сигналу
    port (
        CLK     : in  STD_LOGIC;                     -- тактовий сигнал
        Reset   : in  STD_LOGIC;                     -- сигнал скидання
        DataIn  : in  STD_LOGIC_VECTOR (3 downto 0); -- інформаційний вхід
        DataOut : out STD_LOGIC_VECTOR (3 downto 0); -- інформаційний вихід
```

```
        Command : in  STD_LOGIC_VECTOR (2 downto 0)  -- код команди
    );
end entity lab3;


-- Система команд (варіант 9):
-- 000 - збереження
-- 001 - запис (завантаження DataIn)
-- 010 - перетворення в код: інверсний
-- 011 - зсув ліворуч: циклічний
-- 100 - зсув праворуч: логічний
-- 101 - арифметична операція: +In (додавання DataIn)
-- 110 - логічна операція: nand (з DataIn)
-- 111 - додаткова функція: розворот (реверс розрядів)


architecture lab3 of lab3 is
begin
    process (CLK, Reset)
        variable private : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    begin
        if (Reset = '1') then
            private := "0000";
        elsif (CLK'event) and (CLK = '0') then   -- по задньому фронту
            case Command is
                when "000" => null;                             -- збереження
                when "001" => private := DataIn;                -- запис
                when "010" => private := not private;           -- інверсія
                when "011" =>                                   -- цикл. зсув вліво
                    private := private(2 downto 0) & private(3);
                when "100" =>                                   -- лог. зсув вправо
                    private := '0' & private(3 downto 1);
                when "101" => private := private + DataIn;   -- додавання
                when "110" => private := private nand DataIn; -- nand
                when "111" =>                                   -- розворот
                    private := private(0) & private(1) &
                               private(2) & private(3);
                when others => null;
            end case;
        end if;
        DataOut <= private after Delay;
    end process;
end architecture lab3;
```

TB_LAB3 (тестове оточення):

```
library IEEE;
use IEEE.std_logic_1164.all;


entity tb_lab3 is
end tb_lab3;


architecture tb of tb_lab3 is
    component lab3 is
        generic (Delay : time := 100 ps);
        port (
            CLK     : in  STD_LOGIC;
            Reset   : in  STD_LOGIC;
            DataIn  : in  STD_LOGIC_VECTOR (3 downto 0);
            DataOut : out STD_LOGIC_VECTOR (3 downto 0);
            Command : in  STD_LOGIC_VECTOR (2 downto 0)
        );
    end component;


    signal CLK     : STD_LOGIC := '1';
    signal Reset   : STD_LOGIC := '0';
    signal DataIn  : STD_LOGIC_VECTOR (3 downto 0) := "0000";
    signal DataOut : STD_LOGIC_VECTOR (3 downto 0);
    signal Command : STD_LOGIC_VECTOR (2 downto 0) := "000";
begin
    UUT : lab3 port map (CLK, Reset, DataIn, DataOut, Command);
```

```
    -- тактовий сигнал: період 400 ps (спад на 200, 600, 1000 ... ps)
    clk_gen : process
    begin
        CLK <= '1'; wait for 200 ps;
        CLK <= '0'; wait for 200 ps;
    end process;

    stim : process
        procedure step(cmd : STD_LOGIC_VECTOR(2 downto 0);
                       din : STD_LOGIC_VECTOR(3 downto 0);
                       expected : STD_LOGIC_VECTOR(3 downto 0)) is
        begin
            wait until rising_edge(CLK);      -- команда змінюється при CLK=1
            Command <= cmd;
            DataIn  <= din;
            wait until falling_edge(CLK);     -- виконання по заднього фронту
            wait for 150 ps;                  -- затримка виходу 100 ps
            assert DataOut = expected
                report "Wrong result"
                severity error;
        end procedure;
    begin
        -- початкове скидання
        Reset <= '1'; wait for 100 ps;
        Reset <= '0'; wait for 100 ps;
        step("000", "1011", "0000");   -- збереження (після скидання 0)
        step("001", "1011", "1011");   -- запис 1011 (B)
        step("000", "0000", "1011");   -- збереження
        step("011", "1011", "0111");   -- цикл. зсув ліворуч: 1011 -> 0111
        step("100", "1011", "0011");   -- лог. зсув праворуч: 0111 -> 0011
        step("101", "1011", "1110");   -- додавання: 3 + 11 = 14 (E)
        step("111", "1011", "0111");   -- розворот: 1110 -> 0111
        step("110", "1011", "1100");   -- nand: 0111 nand 1011 = 1100
        step("010", "1011", "0011");   -- інверсія: 1100 -> 0011
        step("101", "1011", "1110");   -- додавання: 3 + 11 = 14
        step("101", "1011", "1001");   -- 14 + 11 = 25 -> 9 (переповнення)
        wait until rising_edge(CLK);
        Reset <= '1'; wait for 250 ps;   -- асинхронне скидання
        assert DataOut = "0000" report "Reset: wrong result" severity error;
        Reset <= '0'; Command <= "000";
        wait for 800 ps;
        report "Simulation finished" severity note;
        wait;
    end process;
end tb;
```

Моделювання:

Для моделювання модулю lab3 встановлено статус Top-Level, виконано ініціалізацію моделювання та створено вікно Waveform із сигналами CLK, Reset, Command, DataIn і DataOut. Тактовий сигнал CLK має період 400 ps (спад на 200, 600, 1000 ps і далі), щоб затримка виходу 100 ps була меншою за півперіод і результат кожної команди було добре видно на діаграмі. Керування регістром здійснюється по задньому фронту синхросигналу, скидання Reset – асинхронне. Команди змінюються при CLK = 1, тобто між робочими фронтами.

Загальна часова діаграма (усі команди):

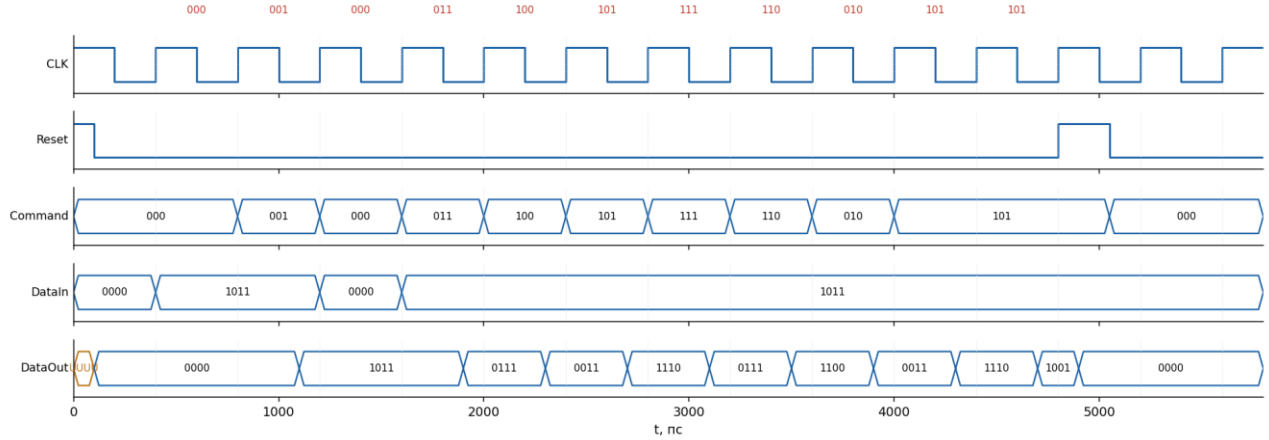


Рисунок 1 – Часова діаграма роботи регістру на всіх командах

Нижче наведено діаграми для кожної команди. Червоний курсор позначає задній фронт CLK, у який виконується команда, синій – момент встановлення результату на виході DataOut через 100 ps.

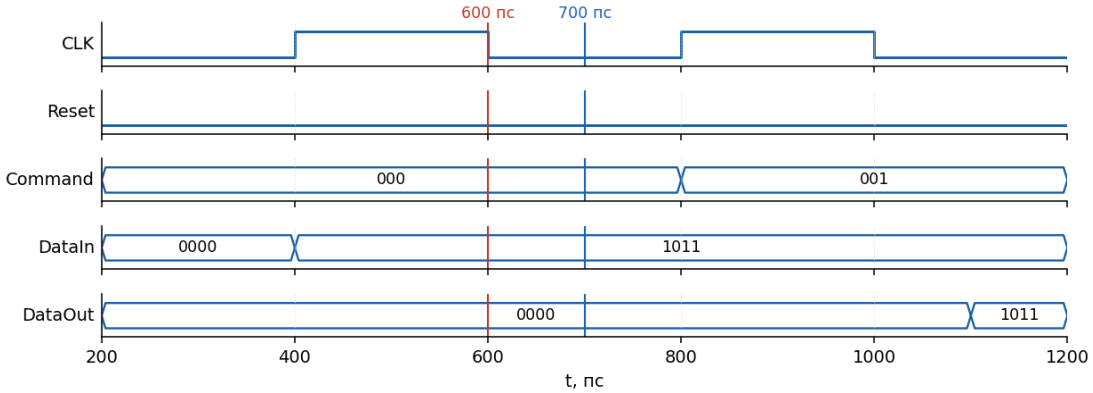


Рисунок 2 – Команда 000 – збереження: стан регістру не змінюється (0000).

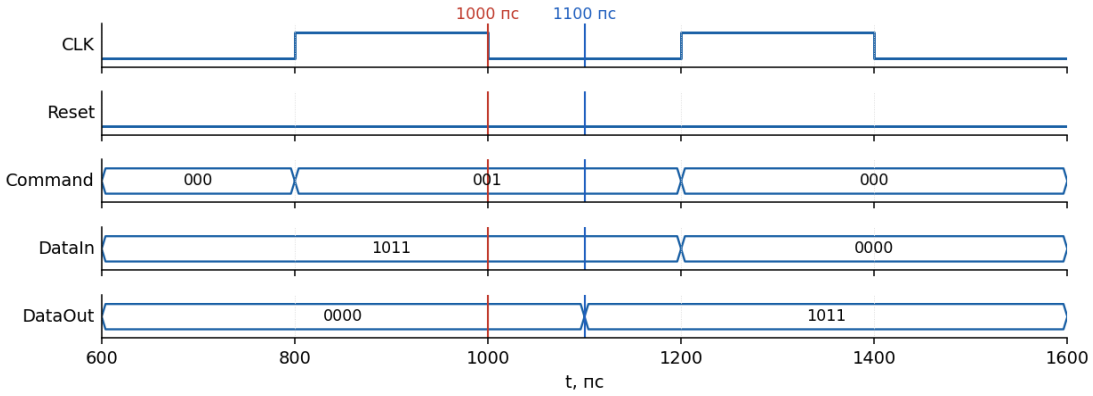


Рисунок 3 – Команда 001 – запис: у регістр завантажується значення DataIn = 1011 (B).

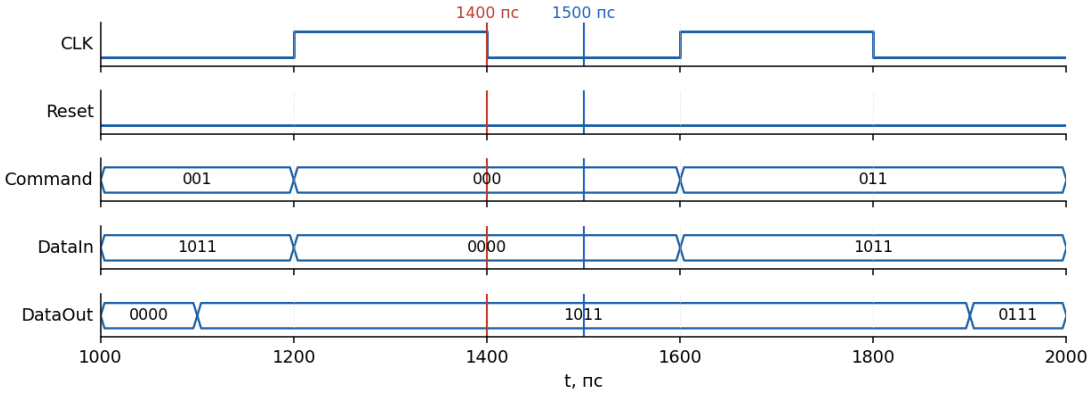


Рисунок 4 – Команда 000 – збереження: значення 1011 зберігається.

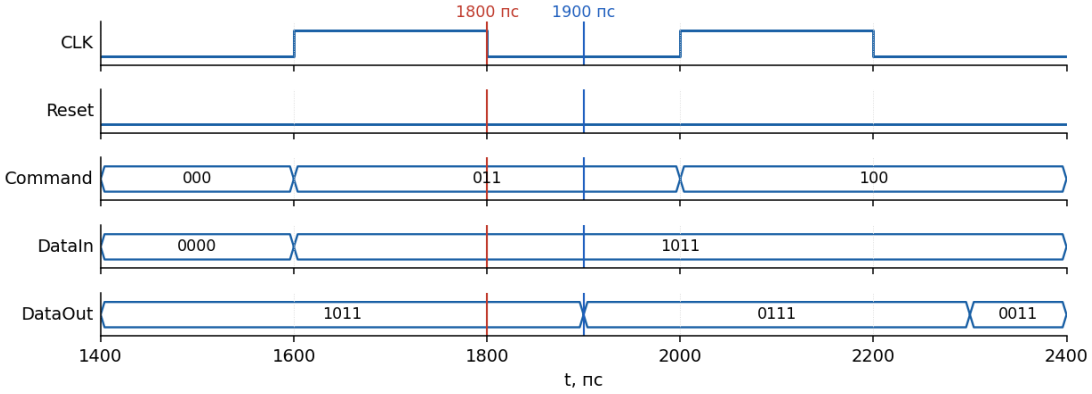


Рисунок 5 – Команда 011 – циклічний зсув ліворуч: 1011 → 0111.

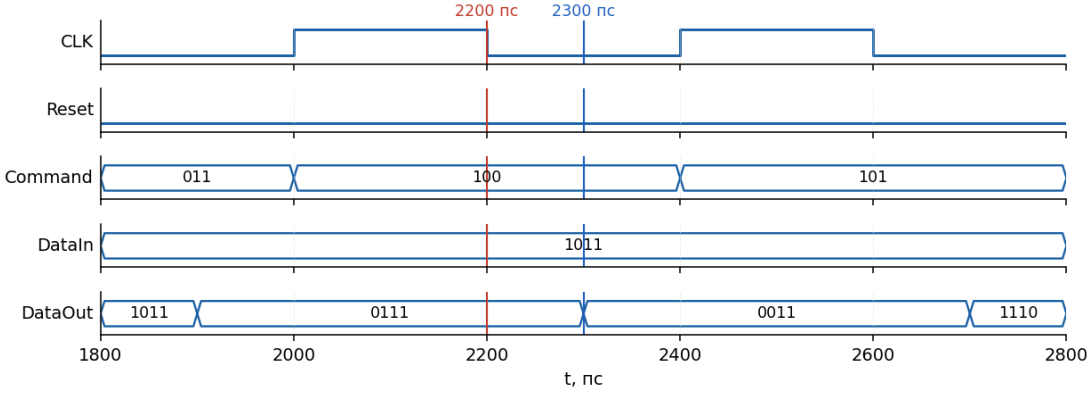


Рисунок 6 – Команда 100 – логічний зсув праворуч: 0111 → 0011.

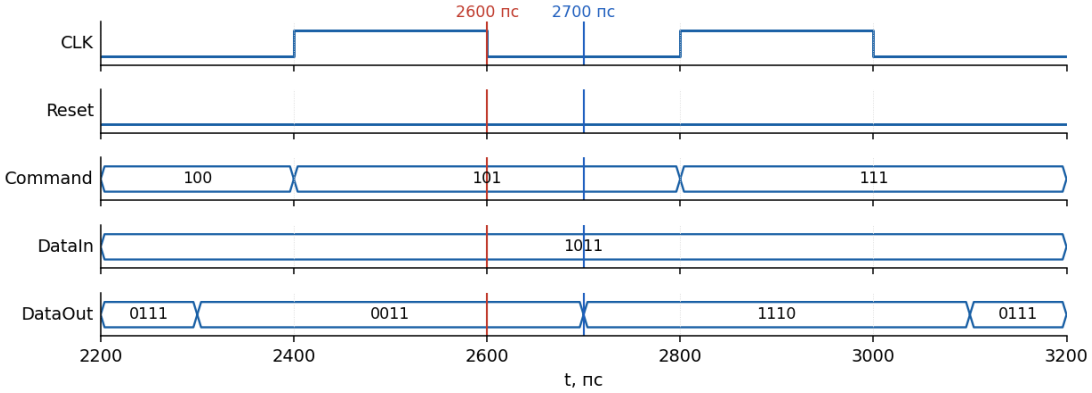


Рисунок 7 – Команда 101 – додавання DataIn: 0011 + 1011 = 1110 (3 + 11 = 14).

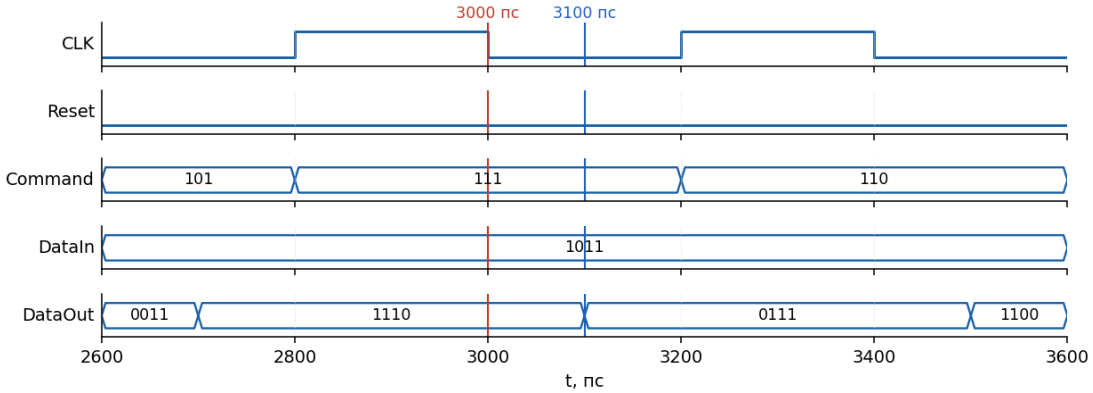


Рисунок 8 – Команда 111 – розворот розрядів: 1110 → 0111.

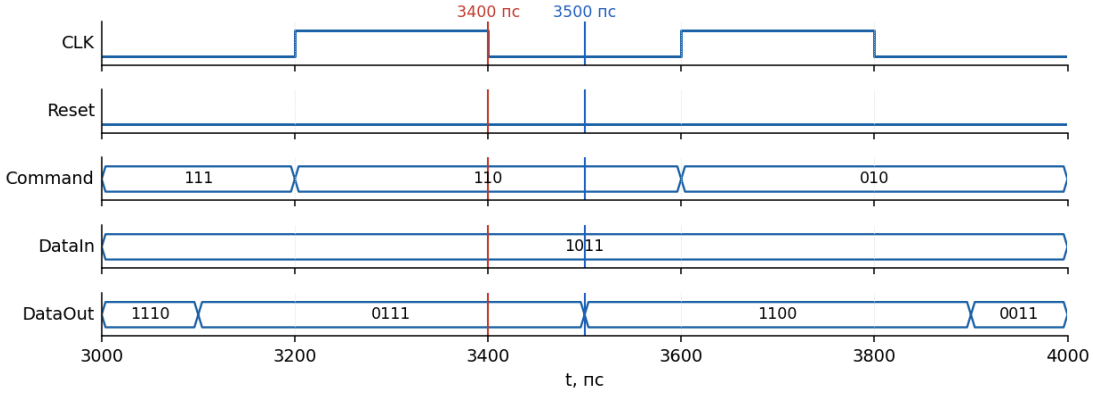


Рисунок 9 – Команда 110 – nand з DataIn: 0111 nand 1011 = 1100.

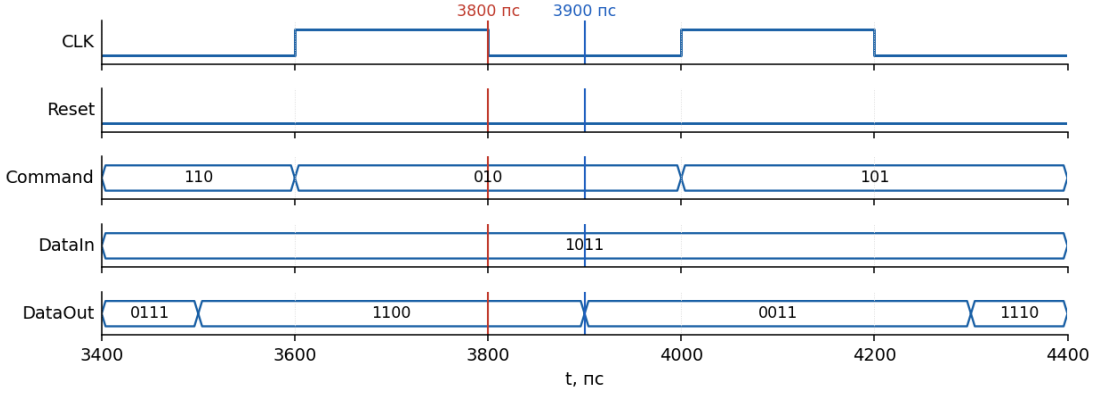


Рисунок 10 – Команда 010 – інверсний код: 1100 → 0011.

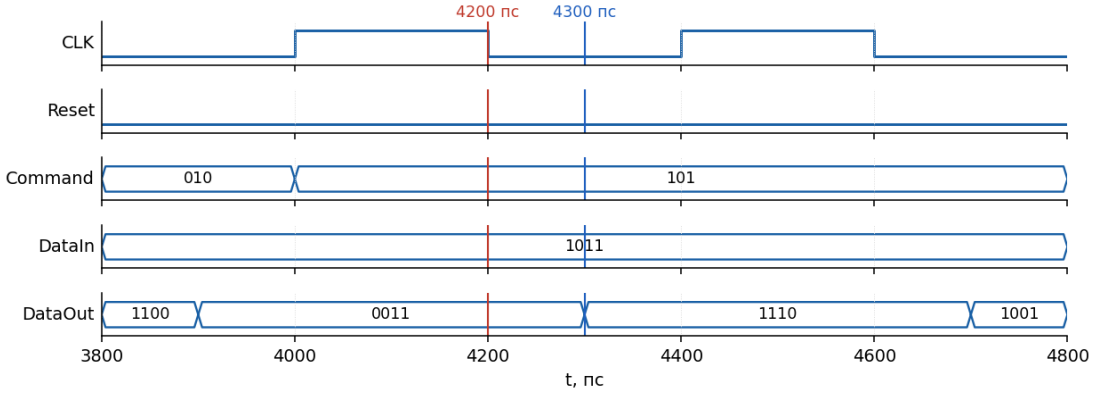


Рисунок 11 – Команда 101 – додавання: 0011 + 1011 = 1110 (3 + 11 = 14).

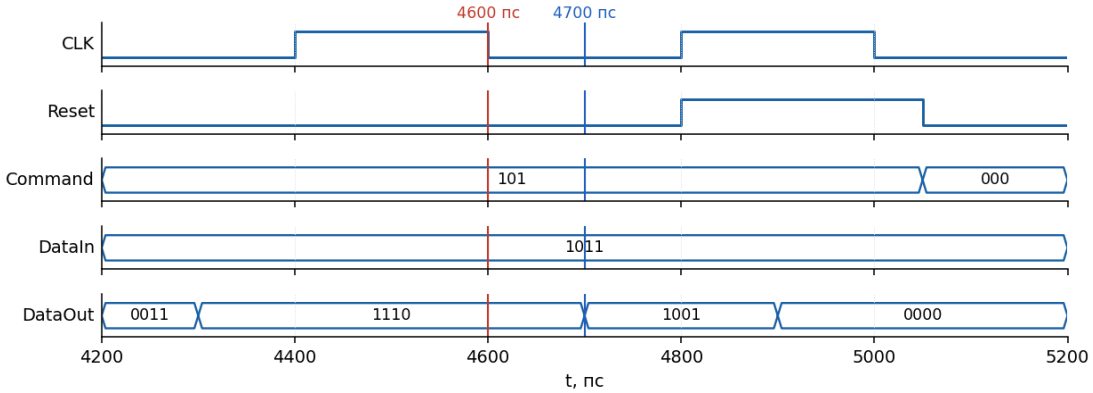


Рисунок 12 – Команда 101 – додавання з переповненням: 1110 + 1011 = 11001 → 1001 (14 + 11 = 25, у 4 розрядах 9).

Результати перевірки:

№	Command	Операція	DataIn	Було	Очікувано	Отримано	Збіг
1	000	збереження	1011	0000	0000	0000	✓
2	001	запис	1011	0000	1011	1011	✓
3	000	збереження	0000	1011	1011	1011	✓
4	011	зсув ліворуч (цикл.)	1011	1011	0111	0111	✓
5	100	зсув праворуч (лог.)	1011	0111	0011	0011	✓
6	101	додавання +DataIn	1011	0011	1110	1110	✓
7	111	розворот	1011	1110	0111	0111	✓
8	110	nand	1011	0111	1100	1100	✓
9	010	інверсний код	1011	1100	0011	0011	✓
10	101	додавання +DataIn	1011	0011	1110	1110	✓
11	101	додавання (переповн.)	1011	1110	1001	1001	✓
12	–	Reset = 1	–	1001	0000	0000	✓

Усі результати збігаються з очікуваними значеннями, перевірка виконана також автоматично за допомогою оператора assert у тестовому оточенні (повідомлень про помилки немає).

Висновок до лабораторної роботи:

У ході лабораторної роботи №3 в середовищі Active-HDL було побудовано та досліджено модель синхронного багатофункціонального регістру розрядністю 4 біти (варіант 9). Модель описана мовою VHDL у вигляді одного процесу, чутливого до сигналів CLK і Reset, зі змінною private для збереження внутрішнього стану та оператором case для вибору команди за 3-розрядним кодом Command. Реалізовано вісім команд: збереження, запис, перетворення в інверсний код, циклічний зсув ліворуч, логічний зсув праворуч, додавання з числом DataIn, логічну операцію nand з DataIn та розворот розрядів. Керування виконується по задньому фронту синхросигналу, скидання – асинхронне, вихідне значення встановлюється із затримкою 100 ps. Під час моделювання перевірено всі команди: часові діаграми та таблиця результатів показали, що вихід DataOut відповідає очікуваним значенням (наприклад, 1011 → 0111 при циклічному зсуві ліворуч, 0111 nand 1011 = 1100, а при додаванні 14 + 11 виникає переповнення і в 4 розрядах зберігається 1001). Також було встановлено, що до першого скидання вихід має невизначене значення U, а сигнал Reset=1 незалежно від команди повертає регістр у стан 0000. Отже, побудований багатофункціональний регістр працює правильно та відповідає заданому варіанту, що підтверджує коректність виконання лабораторної роботи.